CLASE 32

Django Models II - Relaciones

Conectando datos como un profesional

- ForeignKey en profundidad
- ManyToManyField (relaciones muchos a muchos)
- OneToOneField
- QuerySets avanzados (Q objects, annotate)
- Optimización: select_related y prefetch_related
- Evitar el problema N+1

Completar el modelo de datos del Dashboard

Repaso: Nuestros modelos actuales

De la clase anterior:

ForeignKey = Relación 1:N

- Una categoría tiene MUCHOS productos
- Un producto tiene UNA categoría

Hoy agregamos:

- Etiquetas (ManyToMany con Producto)
- Cliente (para las ventas)

ForeignKey — El poder de related_name

Sin related_name

```
producto.categoria
→ Acceso directo al padre

categoria.producto_set.all()
→ Acceso inverso (feo)
```

Con related_name='productos'

```
producto.categoria
→ Acceso directo (igual)

categoria.productos.all()
→ Acceso inverso (limpio)
```

related_name define cómo acceder desde el otro lado de la relación

También funciona en filtros: `Venta.objects.filter(producto__categoria__nombre='Electrónica')`

Siempre usar related_name descriptivo — Convención: plural del modelo relacionado

on_delete — ¿Qué pasa al borrar?

Cuando borras el objeto referenciado, ¿qué pasa con los que lo referencian?

CASCADE	Borrar también los relacionados	*Si borras categoría → borrar productos*
PROTECT	No permitir borrar si tiene relacionados	*No borrar categoría si tiene productos*
SET_NULL	Poner NULL (requiere null=True)	*Producto queda sin categoría*
SET_DEFAULT	Poner valor default	*Mover a categoría 'Sin categoría'*

¿Cuál usar? CASCADE: hijos sin sentido sin padre | PROTECT: prevenir borrados | SET_NULL: hijo puede existir solo

EJERCICIO: Agregar Cliente a Venta

Las ventas necesitan saber QUIÉN compró

Crear modelo Cliente:

nombre → CharField(max_length=100)
email → EmailField(unique=True)
telefono → CharField (opcional)
creado → DateTimeField(auto_now_add)

Modificar Venta — agregar:

```
cliente → ForeignKey(Cliente, on_delete=PROTECT,
related_name='compras')
```

Pasos:

1. Crear modelo Cliente en dashboard_app/models.py
2. Agregar FK cliente a Venta
3. makemigrations (datos existentes)
4. migrate + registrar en admin.py

Tiempo: 8 minutos

ManyToManyField — Muchos a Muchos

Ejemplos del mundo real:

Estudiante	⟷	**Curso**	Un estudiante toma muchos cursos y un curso tiene muchos estudiantes
Producto	⟷	**Etiqueta**	Un producto tiene muchas etiquetas y una etiqueta está en muchos productos
Actor	⟷	**Película**	Un actor actúa en muchas películas y una película tiene muchos actores

¿Cómo funciona? Django crea una tabla intermedia automáticamente:

```
productos_producto_etiquetas:   id  |  producto_id → productos_producto  |  etiqueta_id → productos_etiqueta
```

No necesitas crear la tabla intermedia manualmente

Modelo Etiqueta con ManyToMany

Modelo Etiqueta:

```
class Etiqueta(models.Model):
  nombre = CharField(50, unique)
  color = CharField(7, default)
  def __str__: return self.nombre
```

Agregar a Producto:

```
etiquetas = ManyToManyField(
  Etiqueta,
  blank=True,
  related_name='productos'
)
```

Notas importantes:

- blank=True porque un producto puede no tener etiquetas
- NO hay on_delete (no aplica a ManyToMany)
- related_name funciona igual que en ForeignKey

Implementemos esto juntos — abre productos/models.py

Operaciones con ManyToMany

Agregar

```
producto.etiquetas.add(etiq1, etiq2)
```

Quitar

```
producto.etiquetas.remove(etiq1)
producto.etiquetas.clear()  # todas
```

Establecer

```
producto.etiquetas.set([etiq1, etiq2])
```

Consultar

```
producto.etiquetas.all()
etiqueta.productos.all()
producto.etiquetas.count()
```

.add(), .remove(), .set() modifican la tabla intermedia — no necesitan .save()

ManyToMany con through — Datos extra

A veces necesitas guardar datos EN la relación:

Producto ⟷ Proveedor + precio_compra, fecha_ultimo_pedido, es_principal

Modelo intermedio explícito:

```
class ProductoProveedor(models.Model):
  producto = ForeignKey(Producto)
  proveedor = ForeignKey(Proveedor)
  precio_compra = DecimalField()
  tiempo_entrega = IntegerField()
  es_principal = BooleanField()
```

En el modelo:

```
proveedores = M2MField(
  Proveedor,
  through='ProductoProveedor'
)
```

Usar through cuando necesitas datos extra en la relación. Para etiquetas simples, no es necesario.

OneToOneField — Relación Uno a Uno

Cada objeto A tiene exactamente UN objeto B, y viceversa

User	⟷	**Perfil**	Extender el modelo de usuario de Django
Pedido	⟷	**Factura**	Un pedido tiene exactamente una factura
Producto	⟷	**Detalles**	Separar datos poco usados

Acceso:

```
user.perfil.telefono
perfil.user.email
```

vs ForeignKey:

FK permite muchos del mismo lado
OneToOne es único en AMBOS lados

Es como ForeignKey pero ÚNICO en ambos lados — ideal para extender User

BREAK — 10 MINUTOS

Hasta ahora:

- ForeignKey (related_name, on_delete)
- ManyToManyField (Etiquetas)
- OneToOneField (concepto)
- Cliente agregado a Venta

Volvemos con:

- QuerySets avanzados
- Optimización de queries
- El problema N+1

QuerySets avanzados

Filtros encadenados:

```
Producto.objects
  .filter(activo=True)
  .filter(precio__lte=100)
  .exclude(stock=0)
  .order_by('precio')
```

Acceder a relaciones con __:

```
categoria__nombre='Electrónica'
producto__precio__gte=100
```

Lookups útiles:

`__exact`	Igual (default)
`__icontains`	Contiene (sin mayúsc.)
`__gt, __gte`	Mayor que, mayor o igual
`__lt, __lte`	Menor que, menor o igual
`__in`	En una lista
`__range`	En un rango
`__isnull`	Es NULL

Los QuerySets son lazy: no ejecutan la consulta SQL hasta que realmente los usas

Q objects — OR y consultas complejas

filter() usa AND por defecto — ¿Y si necesitas OR?

Problema:

```
filter(activo=True, precio__lt=100)
→ activo AND precio<100
```

Solución: Q objects

```
from django.db.models import Q
Q(activo=True) | Q(precio__lt=100)
```

Operadores:

```
# OR: baratos O con mucho stock
Q(precio__lt=50) | Q(stock__gte=20)

# AND explícito
Q(activo=True) & Q(precio__lt=100)

# NOT: productos que NO son Electrónica
~Q(categoria__nombre='Electrónica')
```

| (OR) & (AND) ~ (NOT) — Muy útil para búsquedas en múltiples campos

Agregaciones — Cálculos sobre QuerySets

aggregate() — Un resultado:

```
Producto.objects.aggregate(
  total=Count('id'),
  avg=Avg('precio'),
  max=Max('precio')
)
→ {'total': 10, 'avg': 150, 'max': 999}
```

annotate() — Valor por objeto:

```
Categoria.objects.annotate(
  num=Count('productos')
)
→ Cada categoría tiene .num

Producto.objects.annotate(
  vendido=Sum('ventas__cantidad')
).order_by('-vendido')[:5]
```

Útil para el Dashboard:

- Total de ventas del mes
- Producto más vendido
- Promedio de precio por categoría
- Ingresos totales

aggregate = un diccionario resultado | annotate = un valor añadido por cada objeto

El problema N+1 — El asesino del rendimiento

```
El problema:
productos = Producto.objects.all()    # 1 query

for p in productos:
    print(p.categoria.nombre)          # +N queries!

# 100 productos = 101 queries
```

N+1: Ir al super 100 veces,
una por cada producto

Optimizado: Ir una vez
y traer todo

Soluciones: select_related() ⟶ Para ForeignKey y OneToOne (JOIN) | prefetch_related() ⟶ Para ManyToMany y reverse FK

SIEMPRE optimiza cuando accedes a relaciones en loops — puede ser 100x más rápido

select_related y prefetch_related

select_related — FK y O2O:

```
# SIN: 101 queries
Producto.objects.all()

# CON: 1 query (JOIN)
Producto.objects
  .select_related('categoria')
```

prefetch_related — M2M y reverse:

```
# SIN: 101 queries
Producto.objects.all()

# CON: 2 queries
Producto.objects
  .prefetch_related('etiquetas')
```

Combinados:

```
Producto.objects.select_related('categoria').prefetch_related('etiquetas')
```

select_related = JOIN (1 query) | prefetch_related = 2da query separada

Aplicar optimización en views

Antes (N+1):

```
productos = Producto.objects
  .filter(activo=True)
```

Después (optimizado):

```
productos = Producto.objects
  .filter(activo=True)
  .select_related('categoria')
  .prefetch_related('etiquetas')
```

Dashboard optimizado:

```
ventas = Venta.objects.select_related(
    'producto', 'producto__categoria', 'cliente'
)
# Acceder a venta.producto.categoria.nombre = 0 queries extra
```

Regla de oro:

- Si el template accede a relaciones → optimiza
- Mide antes y después con django-debug-toolbar

Resumen — Django Models II

Lo que logramos hoy:

- ForeignKey: related_name y on_delete
- ManyToManyField: Etiquetas para productos
- OneToOneField: concepto y casos de uso
- Cliente agregado al sistema
- QuerySets avanzados: Q objects
- aggregate y annotate
- Problema N+1 identificado y resuelto
- select_related y prefetch_related

PRÓXIMA CLASE: Django Forms

- Formularios HTML automáticos
- ModelForms (forms desde modelos)
- Validación de datos
- CRUD completo con formularios

Dashboard E-commerce:

Setup → URLs → Templates → Models completos ✓